PRACTICAL 6 :Docker Compose for Multi-Container Applications.

- Create a docker-compose.yml file for a multi-container setup (e.g., Flask/FastAPI app + PostgreSQL database.
- Run the application using Docker Compose and test the communication between services.

To create and run a multi-container application using Docker Compose with:

- Flask Application
- PostgreSQL Database

Requirements

- Docker Desktop
- VS Code
- Python 3.x
- Internet connection (for downloading Docker images)

Project Structure

FlaskComposeProject/

```
|
├── app.py
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── .env
```

Step 1: Create Flask Application (app.py)

```
from flask import Flask
import psycopg2
```

```
import os


app = Flask(__name__)


@app.route('/')
def home():
    try:
        conn = psycopg2.connect(
            host=os.getenv("DB_HOST"),
            database=os.getenv("DB_NAME"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD")
        )


        conn.close()


        return "Connected Successfully to PostgreSQL!"


    except Exception as e:
        return "Database Connection Failed: " + str(e)


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 2: Create requirements.txt

```
Flask
psycopg2-binary
```

Step 3: Create Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Step 4: Create .env File

```
DB_HOST=db
DB_NAME=mydatabase
DB_USER=postgres
DB_PASSWORD=postgres
```

Step 5: Create docker-compose.yml

```
version: "3.9"

services:

  web:
    build: .
```

```
  container_name: flask_app

  ports:
    - "5000:5000"

  depends_on:
    - db

  environment:
    DB_HOST: db
    DB_NAME: mydatabase
    DB_USER: postgres
    DB_PASSWORD: postgres

db:
  image: postgres:16

  container_name: postgres_db

  restart: always

  environment:
    POSTGRES_DB: mydatabase
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres

  ports:
    - "5432:5432"
```

```
  volumes:
    - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Part (b): Run the Application Using Docker Compose

Step 1: Open Terminal

Navigate to the project folder.

cd FlaskComposeProject

Step 2: Build the Containers

docker compose build

If successful, Docker builds the Flask image.

Step 3: Start the Containers

docker compose up -d

Expected output:

Creating network "flaskcomposeproject_default"

Creating volume "postgres_data"

Creating postgres_db

Creating flask_app

Step 4: Verify Running Containers

Run:

docker ps

Expected output:

CONTAINER ID IMAGE STATUS

xxxxxxxx flask_app Up

xxxxxxxx postgres:16 Up

Both containers should be running.

Step 5: Test Communication Between Services

Method 1 (Recommended)

Open your browser and visit:

http://localhost:5000

Expected Output:

Connected Successfully to PostgreSQL!

This confirms that:

- Flask is running.
- Flask successfully connected to PostgreSQL using the Docker Compose network.

Method 2 (Check Flask Logs)

docker compose logs web

Expected output:

Running on http://0.0.0.0:5000

Method 3 (Check PostgreSQL Logs)

docker compose logs db

Expected output:

database system is ready to accept connections

Step 6: Stop the Application

docker compose down

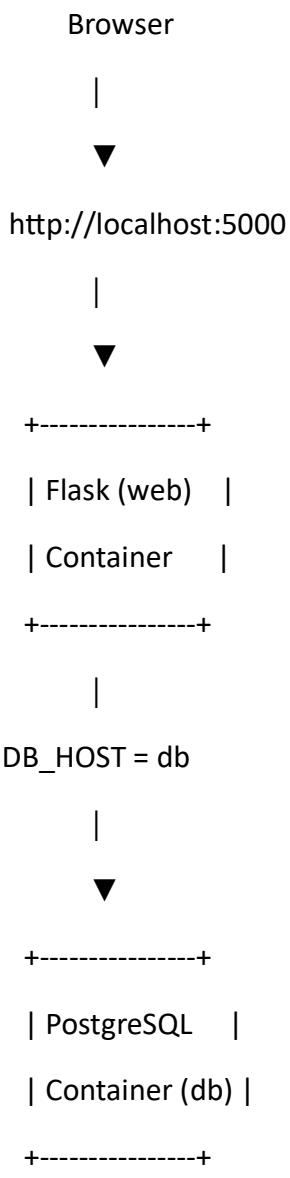
Expected output:

Stopping flask_app

Stopping postgres_db

Removing network

Architecture Diagram



PRAC 7

Yes. For this practical, the **easiest setup is Jenkins on your Windows local machine**, with a simple Flask/FastAPI project in GitHub.

Practical: Setting Up CI/CD with Jenkins

Aim

To set up Jenkins for a Flask/FastAPI application and create a CI/CD pipeline that automatically:

Developer → GitHub → Jenkins → Build → Test → Deploy

Part A — Install Jenkins

1. Install Java

Jenkins requires Java.

You should have **JDK 17 or later** installed.

Open **Command Prompt** and run:

java -version

You should see something similar to:

java version "17..."

If Java is working, continue.

---

2. Install Git

Check Git:

git --version

If you get something like:

git version 2.x.x

Git is installed correctly.

---

3. Install Jenkins

Download Jenkins from the official Jenkins website:

Jenkins official website

For Windows, download the **Windows installer (.msi)**.

During installation:

1. Run the Jenkins installer.
2. Keep the default installation location.
3. Select **LocalSystem** if asked about the service account.
4. Select the Java installation.
5. Keep the default Jenkins port, usually:

8080

6. Complete the installation.

---

4. Open Jenkins

Open your browser and go to:

http://localhost:8080

You should see:

Unlock Jenkins

Jenkins will ask for an initial administrator password.

The password is normally stored here:

C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword

Open that file using Notepad and copy the password.

Paste it into Jenkins.

---

5. Install Jenkins plugins

Select:

Install suggested plugins

Wait until the installation finishes.

Then create your Jenkins administrator account.

You should finally reach the Jenkins dashboard.

Part B — Create a Simple Flask Application

Create a folder:

flask-jenkins-demo

Open it in VS Code.

Create:

flask-jenkins-demo/

```
|
 ├── app.py
 ├── requirements.txt
 └── test_app.py
```

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Flask CI/CD!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
Flask
pytest
```

test_app.py

```
from app import app


def test_home():
    client = app.test_client()
    response = client.get("/")

    assert response.status_code == 200
    assert response.data == b"Hello from Flask CI/CD!"
```

Part C — Test the Application Manually

Open the VS Code terminal.

Create a virtual environment:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Run the test:

```
pytest
```

You should get:

1 passed

Run Flask:

```
python app.py
```

Open:

http://localhost:5000

You should see:

Hello from Flask CI/CD!

Part D — Upload Project to GitHub

Create a new repository on GitHub, for example:

flask-jenkins-demo

Then in your VS Code terminal:

git init

git add .

git commit -m "Initial Flask application"

git branch -M main

git remote add origin YOUR_GITHUB_REPOSITORY_URL

git push -u origin main

After pushing, your GitHub repository should contain:

app.py

requirements.txt

test_app.py

Part E — Create Jenkins Pipeline

Go to:

Jenkins Dashboard

↓

New Item

Enter:

Flask-CI-CD

Select:

Pipeline

Click:

OK

Configure Pipeline

Scroll down to:

Pipeline

Select:

Pipeline script

Enter this Jenkinsfile:

```
pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'YOUR_GITHUB_REPOSITORY_URL'
            }
        }

        stage('Install Dependencies') {
            steps {
                bat 'python -m pip install -r requirements.txt'
            }
        }

        stage('Test') {
            steps {
                bat 'pytest'
            }
```

```
        }

        stage('Deploy') {
            steps {
                bat 'echo Application deployed successfully'
            }
        }
    }
}
```

Replace:

YOUR_GITHUB_REPOSITORY_URL

with your actual GitHub repository URL.

For example:

```
git branch: 'main',
    url: 'https://github.com/username/flask-jenkins-demo.git'
```

Click:

Save

Part F — Run the Jenkins Pipeline

Open your Jenkins job:

Flask-CI-CD

Click:

Build Now

You will see:

Build #1

Click the build number and select:

Console Output

You should see stages such as:

[Pipeline] stage

[Pipeline] { (Checkout)

[Pipeline] stage

[Pipeline] { (Install Dependencies)

[Pipeline] stage

[Pipeline] { (Test)

1 passed

[Pipeline] stage

[Pipeline] { (Deploy)

Application deployed successfully

If everything succeeds, Jenkins will show:

Finished: SUCCESS

Part G — Automatic Build When Code Is Pushed

This is the important part of **CI/CD integration with Git**.

Instead of manually clicking **Build Now**, Jenkins can automatically start whenever you push code to GitHub.

There are two common methods:

Method 1 — GitHub Webhook

The workflow becomes:

Developer

↓

Changes Code

↓

git push

↓

GitHub

↓

Webhook

↓

Jenkins

↓

Checkout

↓

Install

↓

Test

↓

Deploy

Configure Jenkins

Open:

Jenkins

→ Flask-CI-CD

→ Configure

Find:

Build Triggers

Select:

GitHub hook trigger for GITScm polling

Save.

Part H — Configure GitHub Webhook

Go to your GitHub repository.

Select:

Settings

→ Webhooks

→ Add webhook

For a Jenkins server that GitHub can reach, enter your Jenkins webhook endpoint:

http://YOUR_JENKINS_SERVER/github-webhook/

Select:

Content type:

application/json

Choose:

Just the push event

Click:

Add webhook

Important

If Jenkins is running only on your computer at:

localhost:8080

GitHub cannot normally send a webhook directly to localhost.

For a classroom practical, you can demonstrate the pipeline with **Build Now**. For a real automatic webhook, Jenkins needs to be reachable from GitHub, typically through a properly configured public server/tunnel or hosted Jenkins instance.

Part I — Test Automatic CI/CD

Make a change in app.py.

For example:

```
@app.route("/hello")
def hello():
    return "Hello from Jenkins!"
```

Then:

git add .

git commit -m "Added hello endpoint"

git push

GitHub receives the change.

The webhook triggers Jenkins.

Jenkins automatically:

1. Downloads latest code

2. Installs dependencies

3. Runs tests

4. Deploys application

You can see the new build under:

Jenkins → Flask-CI-CD → Build History

What You Need Installed

For this practical, install/check these:

Software	Purpose
JDK 17+	Required by Jenkins
Jenkins	CI/CD automation
Git	Version control
Python	Flask application
VS Code	Development
GitHub account	Remote repository
Flask	Backend application
Pytest	Automated testing

Docker is optional for this basic practical. You don't need Docker just to demonstrate build → test → deploy.

What to Show Your Teacher

For the practical demonstration, show these in order:

1. **Java installed**

java -version

2. **Git installed**

git --version

3. **Jenkins dashboard**

http://localhost:8080

4. **Flask project in VS Code**

5. **GitHub repository**

6. **Jenkins Pipeline configuration**

7. Click **Build Now**

8. Show **Console Output**

9. Show:

1 passed

10. Show:

Finished: SUCCESS

11. Make a Git change and git push.

12. Demonstrate the new Jenkins build.

Final result

Your practical demonstrates all three requirements:

a. Jenkins installed locally ✅
b. Pipeline performs build/install → test → deployment ✅
c. Git/GitHub integrated for automated triggering ✅

Prac 8

Yes. If you are doing this **directly on Windows**, don't use the Ubuntu/Linux commands I gave earlier. We can do the entire practical using **Windows CMD/PowerShell**, without Docker or a VM.

For your practical, I recommend this setup:

Python Flask → Prometheus → Windows Exporter → Grafana

Prometheus provides a Windows windows-amd64 binary, and Grafana officially supports Windows. (Prometheus)

Practical: Monitoring and Observability with Prometheus and Grafana on Windows

What you will install

Software	Purpose	Port
Python + Flask	Application	5000
Python Prometheus Client	Exposes application metrics	8000
Prometheus	Collects/stores metrics	9090
Windows Exporter	CPU & memory metrics	9182
Grafana	Creates dashboards	3000

Part A — Set up Prometheus

Step 1: Download Prometheus for Windows

Go to the official Prometheus download page:

Prometheus Downloads

For a normal 64-bit Windows computer, download:

prometheus-3.14.0.windows-amd64.zip

The current stable release listed by Prometheus is 3.14.0; there is also a newer release candidate, so use the stable release for your practical. (Prometheus)

Extract it, for example:

C:\prometheus

You should see:

C:\prometheus

|

├── prometheus.exe

├── promtool.exe

├── prometheus.yml

└── consoles

Step 2 — Configure Prometheus

Open:

C:\prometheus\prometheus.yml

Open it with Notepad and **replace its contents** with:

```
global:
  scrape_interval: 5s

scrape_configs:

  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "python-app"
    static_configs:
      - targets: ["localhost:8000"]

  - job_name: "windows"
    static_configs:
      - targets: ["localhost:9182"]
```

Save the file.

The important part is:

- job_name: "python-app"

  static_configs:

    - targets: ["localhost:8000"]

This tells Prometheus:

Collect metrics from my Python application's metrics endpoint.

Prometheus works by periodically scraping HTTP metric endpoints from monitored targets. (Prometheus)

Part B — Create the Python Application

Step 3: Create a project folder

Open **CMD**.

mkdir C:\monitoring-app

cd C:\monitoring-app

Create a virtual environment:

python -m venv venv

Activate it:

venv\Scripts\activate

You should see:

(venv) C:\monitoring-app>

Step 4: Install Flask and Prometheus Client

Run:

pip install flask prometheus-client

The official Prometheus Python client is installed with pip install prometheus-client. (Prometheus)

Check:

pip list

You should see:

Flask

prometheus-client

Step 5: Create app.py

Create:

C:\monitoring-app\app.py

Put this code inside:

```
from flask import Flask
from prometheus_client import Counter, start_http_server

app = Flask(__name__)

# Count API requests
REQUEST_COUNT = Counter(
    "api_requests_total",
    "Total number of API requests"
)

@app.route("/")
def home():
    REQUEST_COUNT.inc()
    return "Hello! Flask application is running."

@app.route("/hello")
def hello():
    REQUEST_COUNT.inc()
    return "Hello from the monitoring application!"

if __name__ == "__main__":
```

```
# Prometheus metrics server
start_http_server(8000)

# Flask application
app.run(host="0.0.0.0", port=5000)
```

Step 6: Run the Python Application

In CMD:

cd C:\monitoring-app

venv\Scripts\activate

python app.py

You should see something similar to:

* Running on http://127.0.0.1:5000

Keep this CMD window open.

Step 7: Test Flask

Open your browser:

http://localhost:5000

You should see:

Hello! Flask application is running.

Also test:

http://localhost:5000/hello

Step 8: Test Prometheus Metrics

Open:

http://localhost:8000

You should see Prometheus metrics.

Look for:

api_requests_total

For example:

api_requests_total 2.0

The Python client exposes metrics through an HTTP endpoint; start_http_server(8000) is the simple built-in approach. (Prometheus)

Part C — Start Prometheus

Step 9: Open a Second CMD

Keep your Flask application running.

Open another **CMD** window.

Run:

cd C:\prometheus

Then:

prometheus.exe --config.file=prometheus.yml

You should see Prometheus starting.

Do **not close this CMD window**.

Step 10: Open Prometheus

Go to:

http://localhost:9090

You should see the Prometheus interface.

Go to:

Status

→ Targets

You should see:

prometheus UP

python-app UP

windows UP

Initially, windows will not work until we install Windows Exporter.

Part D — Install Windows Exporter

This is important because you specifically want:

- CPU usage
- Memory consumption

For Windows, use **windows_exporter** rather than Linux Node Exporter. The project provides collectors for Windows CPU, memory, logical disks, network and other system metrics. (GitHub)

Step 11: Download Windows Exporter

Official project:

windows_exporter

Download the Windows installer from the Releases section.

Install the **64-bit Windows version**.

After installation, Windows Exporter normally exposes metrics on:

http://localhost:9182/metrics

Step 12: Test Windows Exporter

Open your browser:

http://localhost:9182/metrics

You should see metrics such as:

windows_cpu_...

windows_memory_...

windows_logical_disk_...

The Windows exporter has CPU and logical-disk collectors enabled by default, with memory available as a collector as well. (GitHub)

Step 13: Check Prometheus Again

Go to:

http://localhost:9090

Then:

Status

→ Targets

You should now have:

prometheus UP

python-app UP

windows UP

This means:

Prometheus is successfully collecting data from your Windows computer and Python application.

Part E — Install Grafana

Step 14: Download Grafana

Use the official Grafana download page:

Grafana Windows Download

Choose:

Windows

→ Windows Installer

→ 64 Bit

Run the .msi installer.

Grafana officially supports Windows and provides a Windows 64-bit installer. (Grafana Labs)

Step 15: Start Grafana

After installation, open:

http://localhost:3000

You should see the Grafana login page.

Log in using the administrator account you created during installation.

Part F — Connect Grafana to Prometheus

Step 16: Add Prometheus Data Source

In Grafana:

Connections

↓

Data sources

↓

Add data source

↓

Prometheus

For URL enter:

http://localhost:9090

Click:

Save & Test

You should receive a successful connection message.

Grafana has built-in Prometheus data-source support. (Grafana Labs)

Part G — Create Dashboard

Go to:

Dashboards

→ New

→ New Dashboard

→ Add visualization

Select:

Prometheus

as the data source.

Now we can create three important panels.

Panel 1 — API Request Rate

Use this PromQL query:

rate(api_requests_total[1m])

Set visualization:

Time series

Panel title:

API Request Rate

This displays approximately how many requests your Flask application receives per second.

Panel 2 — CPU Usage

Because you are on Windows, use Windows Exporter metrics.

First, in the Prometheus query box, search for:

windows_cpu

You can inspect the exact metric names available from your installed exporter.

A common CPU utilization query is:

```
100 - (
  100 *
  avg by (instance) (
    rate(windows_cpu_time_total{mode="idle"}[5m])
  )
)
```

Set:

Visualization: Gauge

Title:

CPU Usage %

Panel 3 — Memory Usage

Search in Prometheus for:

windows_memory

A commonly used query is:

```
100 *
(
 1 -
 windows_memory_available_bytes
 /
 windows_memory_physical_total_bytes
)
```

Set:

Visualization: Gauge

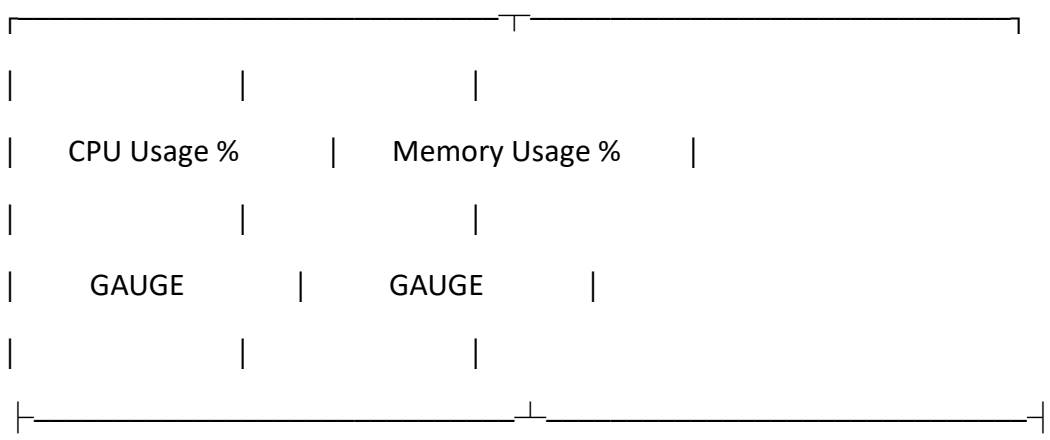
Title:

Memory Usage %

Note: Metric names can vary slightly with the installed windows_exporter version/configuration. If the query returns no data, open http://localhost:9182/metrics and use the exact memory metric names shown there.

Your Final Grafana Dashboard

Create three panels:



| |

| API Request Rate |

| |

| TIME SERIES |

| |

└──┘

Step 17 — Generate API Requests

Go back to your browser:

http://localhost:5000/

Refresh the page several times.

Also visit:

http://localhost:5000/hello

multiple times.

Or use CMD:

curl http://localhost:5000/

Run it several times.

Then Grafana's:

API Request Rate

panel will show the activity.

Complete Windows Setup

You will have **4 CMD windows/processes**:

CMD 1 — Python

cd C:\monitoring-app

venv\Scripts\activate

python app.py

CMD 2 — Prometheus

cd C:\prometheus

prometheus.exe --config.file=prometheus.yml

Windows Exporter

Runs as a **Windows service** after installation.

Grafana

Runs as a **Windows service** after installation.

Then access:

Python:

http://localhost:5000

Python Metrics:

http://localhost:8000

Windows Metrics:

http://localhost:9182/metrics

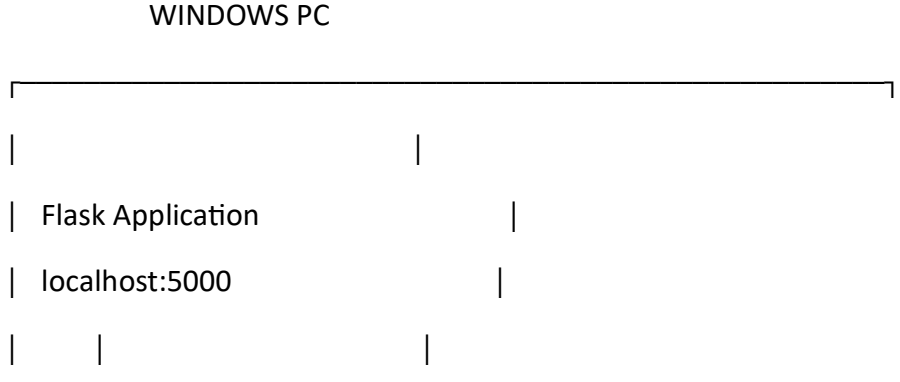
Prometheus:

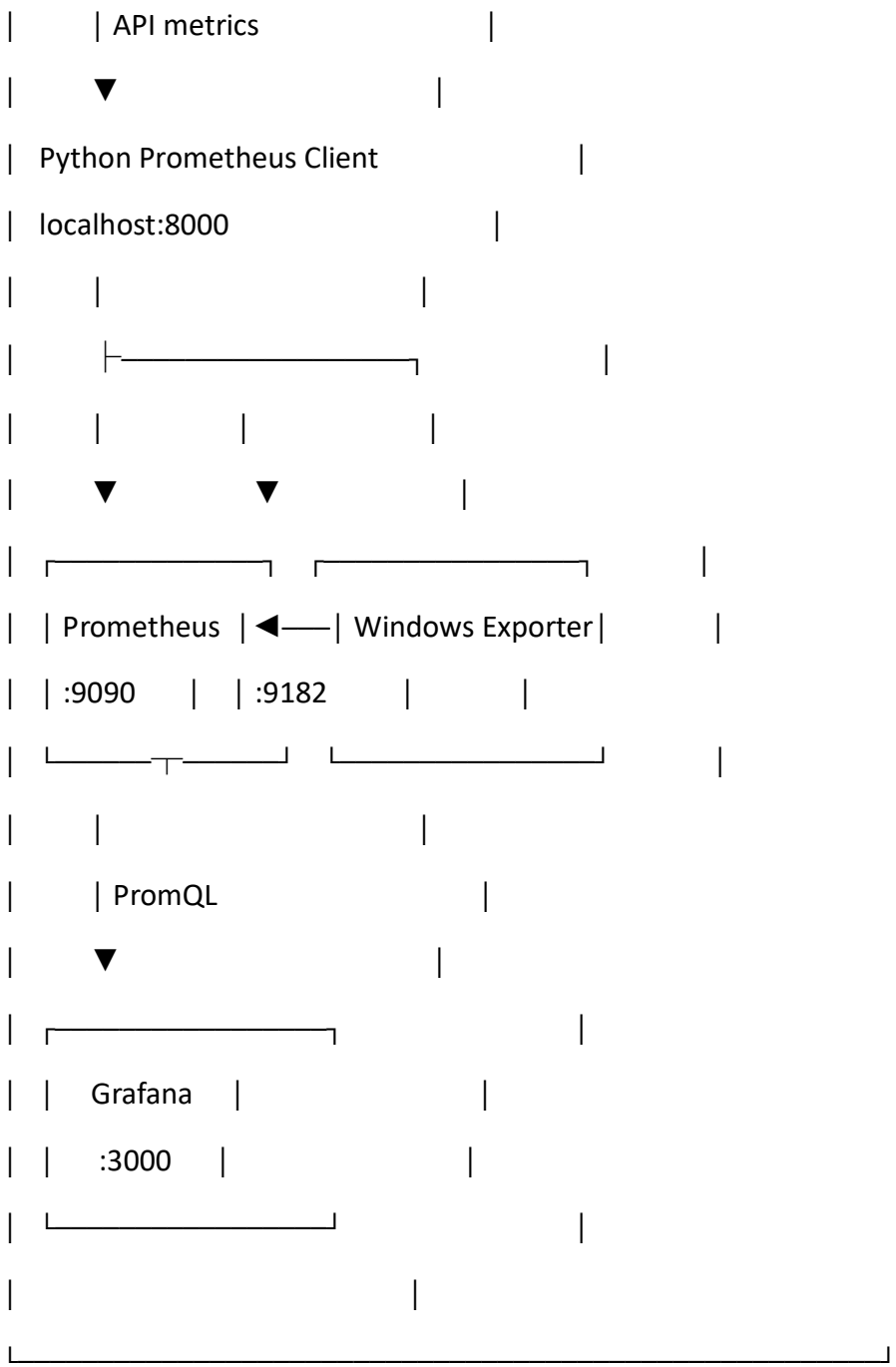
http://localhost:9090

Grafana:

http://localhost:3000

Final architecture



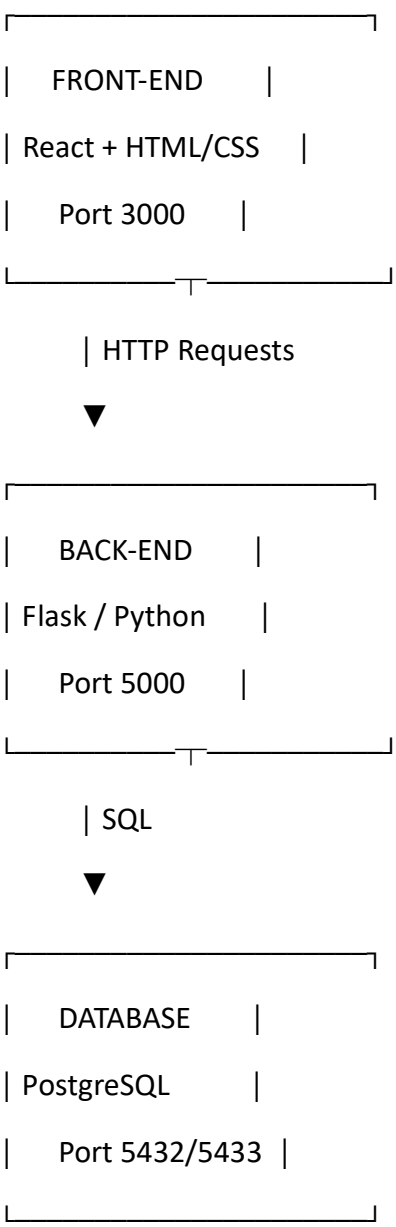


For your college practical, this Windows setup is enough—you do not need Ubuntu, VirtualBox, Docker, or a VM.

PRAC 9

What we are going to build

We will make a simple **Student Management Application**:



The 3 tiers

Tier	**Technology**	**Purpose**
Presentation	React	User interface
Application	Flask	API/business logic
Data	PostgreSQL	Store student data

.

Step 1 — Check what is already installed

Open **Command Prompt** and run these commands **one at a time**:

python --version

node --version

npm --version

For PostgreSQL:

"C:\Program Files\PostgreSQL\18\bin\psql.exe" --version

Also check whether PostgreSQL is using port 5433:

netstat -ano | findstr :5433

Expected result

You should have something similar to:

Python 3.x.x

vxx.x.x

10.x.x

psql (PostgreSQL) 18.x

Since you've already worked through PostgreSQL/pgAdmin setup and your PostgreSQL port was **5433**, we will use:

PostgreSQL → localhost:5433

rather than assuming the default 5432.

Step 2 — Create the project folder

Open **Command Prompt** and run:

cd %USERPROFILE%\Desktop

Then:

mkdir three-tier-student-app

Then:

cd three-tier-student-app

Now create the three tiers:

mkdir backend

mkdir frontend

Your structure will be:

three-tier-student-app

|

├── backend

|

└── frontend

Step 3 — Create the PostgreSQL database

Open **pgAdmin**.

Go to:

Servers → PostgreSQL 18 → Databases

Right-click **Databases** → **Create → Database**

Use:

Database: studentdb2

Then click **Save**.

Step 4 — Create the students table

In pgAdmin, select:

studentdb2 → Query Tool

Run:

```
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    course VARCHAR(100) NOT NULL
);
```

Then click the **Execute** ▶ button.

You should get:

Query returned successfully

Then insert some sample data:

```
INSERT INTO students (name, email, course)
VALUES
('Rahul', 'rahul@gmail.com', 'BCA'),
('Priya', 'priya@gmail.com', 'BSc CS'),
('Amit', 'amit@gmail.com', 'BCA');
```

Run it.

Now your database tier is ready:

```
studentdb2
  |
  └── students
      ├── id
      ├── name
      ├── email
      └── course
```

Database tier is complete.

Now we will build the **Backend Tier using Flask** and connect it to your PostgreSQL database studentdb2 on **port 5433**.

Step 5 — Create the Flask backend

Open **Command Prompt**.

If you closed it, go back to your project:

```
cd %USERPROFILE%\Desktop\three-tier-student-app
```

Enter the backend folder:

```
cd backend
```

5.1 Create a virtual environment

Run:

python -m venv venv

Activate it:

venv\Scripts\activate

You should now see something like:

(venv) C:\Users\192\Desktop\three-tier-student-app\backend>

5.2 Install Flask and PostgreSQL connector

Run:

python -m pip install flask flask-cors "psycopg[binary]"

Wait until the installation finishes.

Then verify Flask:

python -c "import flask, psycopg; print('Flask and PostgreSQL connector OK')"

You should see:

Flask and PostgreSQL connector OK

Step 6 — Create app.py

Inside the backend folder, create a file named:

app.py

Open the entire project folder in VS Code:

code .

If code is not recognized, simply open VS Code manually and open:

Desktop

→ three-tier-student-app

→ backend

Put this code inside **app.py**:

from flask import Flask, jsonify, request

from flask_cors import CORS

import psycopg

```
app = Flask(__name__)
CORS(app)

# PostgreSQL connection
DB_CONFIG = {
    "host": "localhost",
    "port": 5433,
    "dbname": "studentdb2",
    "user": "postgres",
    "password": "YOUR_POSTGRES_PASSWORD"
}

def get_connection():
    return psycopg.connect(**DB_CONFIG)

@app.route("/")
def home():
    return jsonify({
        "message": "Student Management API is running"
    })

@app.route("/students", methods=["GET"])
def get_students():
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        "SELECT id, name, email, course FROM students ORDER BY id"
```

```
    )

    students = cur.fetchall()

    cur.close()
    conn.close()

    result = []

    for student in students:
        result.append({
            "id": student[0],
            "name": student[1],
            "email": student[2],
            "course": student[3]
        })

    return jsonify(result)

@app.route("/students", methods=["POST"])
def add_student():
    data = request.get_json()

    name = data.get("name")
    email = data.get("email")
    course = data.get("course")

    conn = get_connection()
```

```
    cur = conn.cursor()

    cur.execute(
        """
        INSERT INTO students (name, email, course)
        VALUES (%s, %s, %s)
        RETURNING id
        """,
        (name, email, course)
    )

    student_id = cur.fetchone()[0]

    conn.commit()

    cur.close()
    conn.close()

    return jsonify({
        "message": "Student added successfully",
        "id": student_id
    }), 201

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

Important: change the password

Find this line:

"password": "YOUR_POSTGRES_PASSWORD"

Replace YOUR_POSTGRES_PASSWORD with the **password you created when installing PostgreSQL**.

For example, if your PostgreSQL password is postgres123:

"password": "postgres123"

Do **not** change these:

"host": "localhost",

"port": 5433,

"dbname": "studentdb2",

"user": "postgres",

because they match your current setup.

Step 7 — Start the Flask backend

Go back to Command Prompt.

Make sure you are still inside:

three-tier-student-app\backend

and that (venv) is visible.

Run:

python app.py

You should see approximately:

* Serving Flask app 'app'

* Debug mode: on

* Running on http://127.0.0.1:5000

Test the backend

Open your browser and visit:

http://127.0.0.1:5000/

You should get:

{

"message": "Student Management API is running"

}

Then open:

http://127.0.0.1:5000/students

You should see your students:


```
[
 {
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "course": "BCA"
 },
 {
  "id": 2,
  "name": "Priya",
  "email": "priya@gmail.com",
  "course": "BSc CS"
 },
 {
  "id": 3,
  "name": "Amit",
  "email": "amit@gmail.com",
  "course": "BCA"
 }
]
```


Your application now looks like this

React Frontend

↓

Flask API

↓

PostgreSQL

studentdb2

↓

students table

Your PostgreSQL connector is fixed.

Now let's **test the Flask backend connection to PostgreSQL** before creating the React frontend.

Step 8 — Start Flask

In the same Command Prompt, make sure you see (venv) and run:

python app.py

You should see:

* Serving Flask app 'app'

* Debug mode: on

* Running on http://127.0.0.1:5000

Step 9 — Test the Flask API

Keep that Command Prompt **running**.

Open your browser and go to:

http://127.0.0.1:5000/

You should see:

{

"message": "Student Management API is running"

}

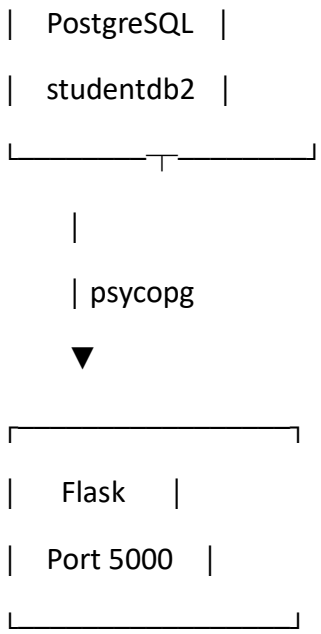
Then open:

http://127.0.0.1:5000/students

You should see the students you inserted earlier.

If /students works.Your Backend + Database tiers are successfully connected:

┌──────────────────────



Do this now and tell me what you see at /students. Then we'll build the **React frontend (Tier 1)**.

Perfect! 🎉 Your **Flask backend and PostgreSQL database are working correctly**.

Now we build the **Frontend Tier using React**.

Step 10 — Create the React frontend

Keep the Flask Command Prompt running. Open a **new Command Prompt** window.

Go to your project folder:

cd %USERPROFILE%\Desktop\three-tier-student-app

Now create the React application:

npm create vite@latest frontend -- --template react

If it asks:

Need to install the following packages...

Ok to proceed? (y)

Type:

y

Wait until it finishes.

Step 11 — Install React dependencies

Run:

cd frontend

Then:

npm install

Then:

npm install axios

Step 12 — Replace the React code

Open the project in VS Code:

code .

In VS Code, open:

frontend

└── src

    └── App.jsx

Delete everything in App.jsx and paste:

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [students, setStudents] = useState([]);

  useEffect(() => {
    axios
      .get("http://127.0.0.1:5000/students")
      .then((response) => {
        setStudents(response.data);
      })
      .catch((error) => {
```

```
      console.error("Error fetching students:", error);
    });
}, []);

return (
  <div className="container">
    <h1>Student Management System</h1>

    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
          <th>Course</th>
        </tr>
      </thead>

      <tbody>
        {students.map((student) => (
          <tr key={student.id}>
            <td>{student.id}</td>
            <td>{student.name}</td>
            <td>{student.email}</td>
            <td>{student.course}</td>
          </tr>
        ))}
      </tbody>
```

```
      </table>
    </div>
  );
}

export default App;
```

Step 13 — Add simple CSS

Open:

frontend
 └── src
     └── App.css

Delete its existing contents and paste:

```
body {
  font-family: Arial, sans-serif;
  background: #f2f2f2;
  margin: 0;
  padding: 40px;
}

.container {
  max-width: 900px;
  margin: auto;
  background: white;
  padding: 30px;
  border-radius: 10px;
}
```

```
h1 {
  text-align: center;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 25px;
}

th,
td {
  border: 1px solid #ccc;
  padding: 12px;
  text-align: left;
}

th {
  background: #eeeeee;
}
```

Save both files.

Step 14 — Start React

Go back to your **second Command Prompt**.

Make sure you are inside:

three-tier-student-app\frontend

Run:

npm run dev

You should get something similar to:

VITE ready

Local: http://localhost:5173/

Open that address in your browser:

http://localhost:5173/

🎉 **Expected result**

You should see:

Student Management System

ID Name Email Course

---

1 Rahul rahul@gmail.com BCA

2 Priya priya@gmail.com BSc CS

3 Amit amit@gmail.com BCA

The important part is that these students are **not hard-coded in React**.

The data is flowing:

React

|

| HTTP GET

▼

Flask API

|

| SQL

▼

PostgreSQL

|

| Student data

▼

Flask

|

▼

React

That means your **3-tier architecture is working**. ✅

PRAC 10

Part A: Dockerize Each Component and Use Docker Compose

Step 1: Create Project Folder

Open Command Prompt or PowerShell:

cd %USERPROFILE%\Desktop

mkdir three-tier-docker

cd three-tier-docker

Create the folders:

mkdir backend

mkdir frontend

mkdir database

The structure will be:

three-tier-docker/

|

├── backend/

├── frontend/

├── database/

└── docker-compose.yml

Step 2: Create Flask Backend

Go into the backend folder:

cd backend

Create:

app.py

Add:

```
from flask import Flask, jsonify
import psycopg2
import os
```

```
app = Flask(__name__)

@app.route("/")
def home():
    return "Flask Backend is Running"

@app.route("/students")
def students():
    try:
        conn = psycopg2.connect(
            host=os.getenv("DB_HOST"),
            database=os.getenv("DB_NAME"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD")
        )

        cur = conn.cursor()

        cur.execute("SELECT id, name, email, course FROM students ORDER BY id")

        rows = cur.fetchall()

        cur.close()
        conn.close()

        result = []
```

```
        for row in rows:
            result.append({
                "id": row[0],
                "name": row[1],
                "email": row[2],
                "course": row[3]
            })

        return jsonify(result)

    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Create Backend requirements.txt

Inside the backend folder, create:

requirements.txt

Add:

```
Flask
psycopg2-binary
flask-cors
```

Step 4: Create Backend Dockerfile

Inside backend, create a file named:

Dockerfile

Add:

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

The backend is now ready to be containerized.

Step 5: Create React Frontend

Go back to the main project folder:

cd ..

Go into frontend:

cd frontend

Create a React application:

npm create vite@latest . -- --template react

When asked to continue, type:

y

Then install the packages:

npm install

Install Axios:

npm install axios

Step 6: Modify React App

Open:

frontend/src/App.jsx

Replace the existing code with:

```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {

 const [students, setStudents] = useState([]);

 useEffect(() => {

  axios
   .get("http://localhost:5000/students")
   .then((response) => {
    setStudents(response.data);
   })
   .catch((error) => {
    console.error(error);
   });

 }, []);

 return (
  <div>
   <h1>Student Management System</h1>
```

```
<table border="1" cellPadding="10">

 <thead>
  <tr>
   <th>ID</th>
   <th>Name</th>
   <th>Email</th>
   <th>Course</th>
  </tr>
 </thead>

 <tbody>

  {students.map((student) => (

   <tr key={student.id}>

    <td>{student.id}</td>
    <td>{student.name}</td>
    <td>{student.email}</td>
    <td>{student.course}</td>

   </tr>

  ))}

 </tbody>
```

```
      </table>

    </div>
  );
}

export default App;
```

Step 7: Create Frontend Dockerfile

Inside the frontend folder, create:

Dockerfile

Add:

```
FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

Step 8: Configure PostgreSQL

We will use PostgreSQL as a Docker container.

Create a folder:

database

Inside it, create:

init.sql

Add:

```
CREATE TABLE IF NOT EXISTS students (
   id SERIAL PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(100) NOT NULL,
   course VARCHAR(100) NOT NULL
);


INSERT INTO students (name, email, course)
VALUES
('Rahul', 'rahul@gmail.com', 'BCA'),
('Priya', 'priya@gmail.com', 'BSc CS'),
('Amit', 'amit@gmail.com', 'BCA');
```

This SQL file will automatically create the table when the PostgreSQL container is initialized.

Step 9: Create docker-compose.yml

Go back to the main project folder:

cd ..

Create:

docker-compose.yml

Add:

```
services:
```

```
frontend:
  build: ./frontend
  container_name: student_frontend
  ports:
    - "5173:5173"
  depends_on:
    - backend

backend:
  build: ./backend
  container_name: student_backend
  ports:
    - "5000:5000"
  environment:
    DB_HOST: database
    DB_NAME: studentdb
    DB_USER: postgres
    DB_PASSWORD: postgres
  depends_on:
    - database

database:
  image: postgres:16
  container_name: student_database
  restart: always
  environment:
    POSTGRES_DB: studentdb
```

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:

postgres_data:

Important

Inside Docker Compose, the Flask application does not use:

localhost

to connect to PostgreSQL.

It uses:

database

because database is the PostgreSQL service name.

Step 10: Final Project Structure

Your project should now look like:

three-tier-docker/

|

├── backend/

| ├── app.py

| ├── requirements.txt

| └── Dockerfile

|

├── frontend/

| ├── src/

| | └── App.jsx

| ├── package.json

| ├── package-lock.json

| └── Dockerfile

|

├── database/

| └── init.sql

|

└── docker-compose.yml

Step 11: Build All Containers

Open the terminal in:

three-tier-docker

Run:

docker compose build

Docker will build:

- **React frontend image**
- **Flask backend image**
- **PostgreSQL will be downloaded from Docker Hub**

Step 12: Start the Application

Run:

docker compose up -d

The -d option runs the containers in the background.

You should see containers being created.

Step 13: Check Running Containers

Run:

docker ps

You should see three containers similar to:

student_frontend

student_backend

student_database

All three should have status:

Up

Step 14: Check Docker Compose Services

Run:

docker compose ps

Expected:

NAME STATUS

student_frontend Up

student_backend Up

student_database Up

Step 15: Test the Backend

Open a browser and visit:

http://localhost:5000/

Expected output:

Flask Backend is Running

Now visit:

http://localhost:5000/students

Expected output:

[

{

```
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "course": "BCA"
 },
 {
  "id": 2,
  "name": "Priya",
  "email": "priya@gmail.com",
  "course": "BSc CS"
 },
 {
  "id": 3,
  "name": "Amit",
  "email": "amit@gmail.com",
  "course": "BCA"
 }
]
```


This proves that:

Flask → PostgreSQL

communication is working.

Step 16: Test the Frontend

Open:

http://localhost:5173

You should see:

Student Management System

ID Name Email Course

----

1 Rahul rahul@gmail.com BCA

2 Priya priya@gmail.com BSc CS

3 Amit amit@gmail.com BCA

The complete communication is:

React

|

| HTTP Request

▼

Flask

|

| SQL

▼

PostgreSQL

|

| Student Data

▼

Flask

|

▼

React

Part B: Deploy on a Virtual Machine or Local Setup

For the practical, you can demonstrate the application on your local computer or an Ubuntu virtual machine.

Option 1: Local Setup

Docker Desktop must be running.

Run:

docker compose up -d

Then open:

http://localhost:5173

If the Student Management System appears with student records, the deployment is successful.

Option 2: Ubuntu Virtual Machine

Copy the project to the Ubuntu VM.

Install Docker if required.

Then go to the project folder:

cd three-tier-docker

Build:

docker compose build

Start:

docker compose up -d

Check:

docker ps

Find the VM's IP address:

hostname -I

For example:

192.168.1.20

If the VM is configured for network access, open from another computer:

http://192.168.1.20:5173

Step 17: View Container Logs

To check the frontend:

docker compose logs frontend

Backend:

docker compose logs backend

Database:

docker compose logs database

For the PostgreSQL container, you should eventually see a message indicating that the database is ready to accept connections.

Step 18: Stop the Application

When testing is complete:

docker compose down

This stops and removes the containers and network.

The PostgreSQL volume remains, so database data can persist.

Step 19: Restart the Application

To start it again:

docker compose up -d

Check:

docker compose ps

Then open:

http://localhost:5173

Docker Commands Summary

Command	Purpose
docker compose build	Build application images
docker compose up -d	Start all services
docker compose ps	Display service status
docker ps	Display running containers

Command	Purpose
docker compose logs frontend	View frontend logs
docker compose logs backend	View backend logs
docker compose logs database	View database logs
docker compose down	Stop and remove containers
docker compose up -d	Restart the application

Expected Result

The 3-Tier application is successfully deployed using Docker Compose.

The final setup contains:

Browser

|

▼

React Frontend

Docker Container

Port 5173

|

▼

Flask Backend

Docker Container

Port 5000

|

▼

PostgreSQL Database

Docker Container

Port 5432

Result

The frontend, backend, and database components were successfully Dockerized and orchestrated using Docker Compose. The 3-Tier application was deployed on the local/virtual machine environment, and communication between React, Flask, and PostgreSQL was successfully tested.